HLD

Credit91 - High-Level Design

Credit91 -- Loan Origination & Management System

Next.js 16 + TypeScript + Supabase (Postgres / Auth / Storage) + Vercel

1. Overview

Credit91 is a Loan Origination System (LOS) and Loan Management System (LMS) built as a single Next.js application backed by Supabase. It covers the full lifecycle of an unsecured loan: an applicant submits a loan application, an underwriter reviews and decides on it, and on acceptance a loan is disbursed with a generated repayment schedule that the applicant pays down over time, including an admin-facing collections and interest-accrual workflow for loans that fall behind.

Constraints pitch: Given assessment time and cost constraints, external integrations (credit bureau, payment gateway, notifications) were mocked and the scheduled interest-accrual job was implemented as an admin-triggered action performing identical calculation logic; in production these would be replaced with a real bureau API, payment webhook handler, and a cron-based worker (e.g. Vercel Cron or a queue consumer) with no change to the core domain logic. Email confirmation was disabled to stay within Supabase's free-tier email rate limits; in production this would go through a transactional email provider (e.g. Resend, SES) with confirmation required before first login.

2. Architecture

Stack

Layer	Choice	Rationale
App	Next.js 16 (App Router, TypeScript)	One codebase for pages + API routes; no CORS, no separate backend
DB / Auth / Storage	Supabase (Postgres, Auth, Storage)	One free account replaces 3 separate services
Hosting	Vercel	Deploys the Next.js app directly from GitHub
Interest accrual	Admin-triggered 'Run Accrual' action	Same calculation a scheduled job would run, without cron setup
Credit check / payments	Mocked API routes	No external accounts, keys, or cost

System diagram

The browser talks only to the Next.js app. Server Components and Route Handlers use a Supabase client authenticated with the caller's session cookie, so every query is evaluated under Postgres Row Level Security -- there is no separate authorization layer to keep in sync with the database.

3. Roles & Security Model

There are exactly two roles, stored on a profiles.role column populated at signup: applicant and underwriter. A Postgres trigger (handle_new_user) creates the profiles row automatically from signup metadata, so the application code never has to synchronize roles by hand.

- Applicants can only see and act on their own applications, documents, credit checks, loans, and payments.
- Underwriters can read every application, run decisions, and view the full audit log and collections list.
- Both the database (Row Level Security policies) and the API routes enforce this -- the API layer checks profile.role before allowing an action, and RLS enforces it again at the data layer, so a bug in one layer doesn't expose data.

4. Data Flow

Applicant journey

Sign up (role=applicant) -> submit application (income, employment, amount, tenure) -> upload ID proof document -> trigger mock credit check -> wait for underwriter decision -> if approved, accept offer (creates Loan + amortization schedule, instantly 'disbursed') -> view repayment schedule -> Simulate Payment against the next due installment.

Underwriter journey

Sign up (role=underwriter) -> see all submitted applications with credit score -> open an application -> approve (set amount/rate/tenure) or reject (with reason) -> Collections screen lists loans with installments past their due date -> Run Accrual applies a late-payment penalty and flags the loan delinquent.

5. Scope Cuts

In scope

- Signup/login with applicant + underwriter roles
- Application form and status tracking
- Document upload (Supabase Storage, one file type)
- Mocked credit score generator
- Underwriter approve/reject with custom rate & tenure
- Loan creation + reducing-balance amortization schedule
- Simulated repayments and balance reduction
- Collections list + manual interest-accrual trigger
- Audit log on every key action
- Row Level Security on every table

Out of scope (deliberate cuts)

- Escrow management
- Real payment gateway / webhook handling
- Real credit bureau integration
- Email/SMS notifications (including signup confirmation email)
- Multi-document KYC / OCR verification
- Fine-grained RBAC beyond the two roles
- A real scheduled cron job for interest accrual

6. Deployment View

The application is a single Vercel project built from the GitHub repository main branch. Two environment variables (NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY) connect it to the Supabase project. The Supabase schema, RLS policies, and storage bucket are provisioned by a single SQL migration file (supabase/migrations/0001_init.sql) that is run once via the Supabase SQL Editor. A seed script (scripts/seed.ts) can populate demo accounts and loans in various states for a live walkthrough.

Live URL: [add your Vercel deployment URL here once deployed]